

what the bits mean

digitalisierung und programmierung · prof. dr. nicolas meseth

- 1 the agreement is everything
- 2 build your own codebook
- 3 where does a character end?
- 4 the same bits, other meanings
- 5 codes without computers

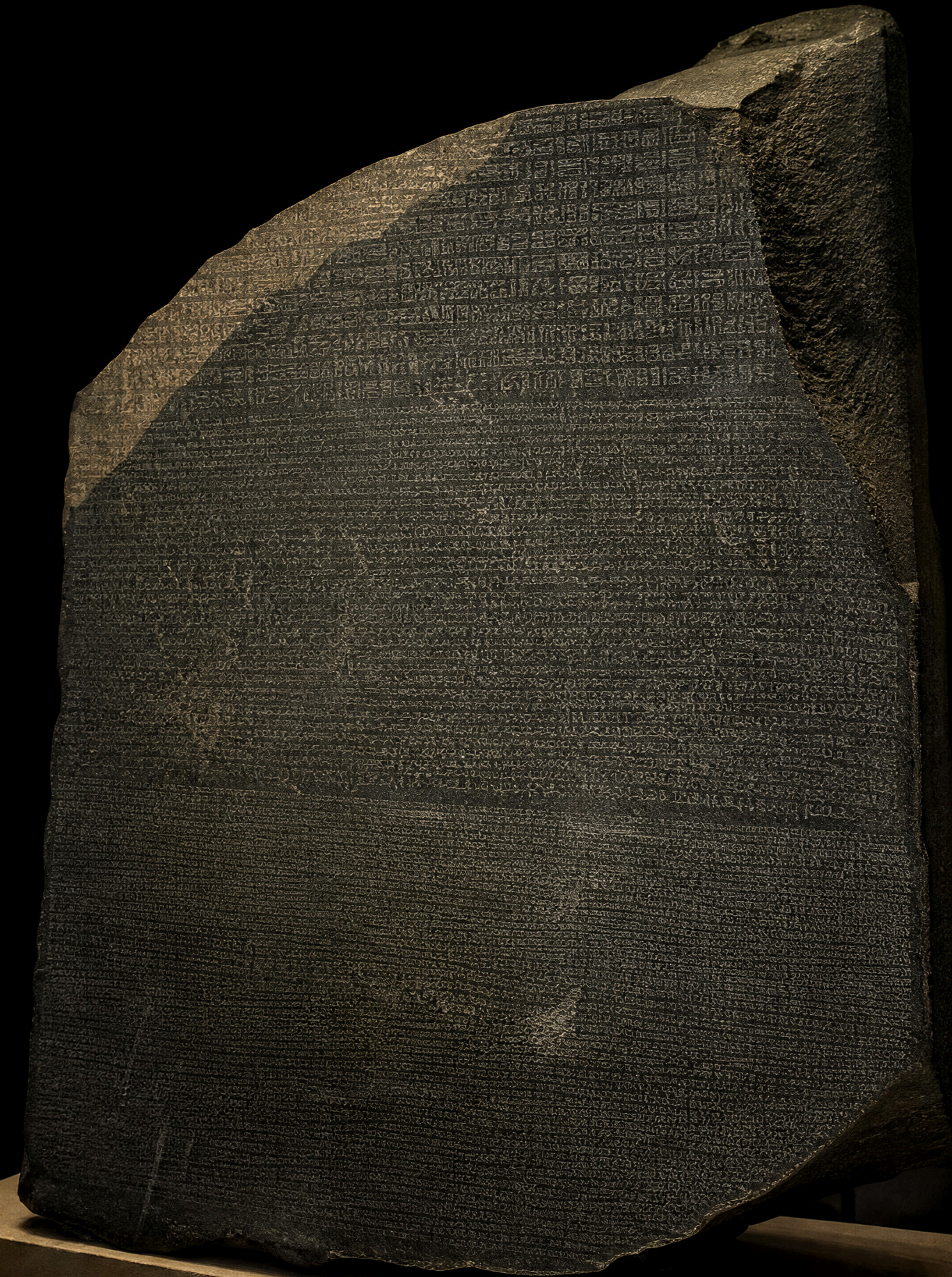
part 1

the agreement is everything

who still knows
what this
means?

image: ai-generated (gpt-image-2)

the agreement is everything



what a code system is



a code is not encryption, not compression, and not source code

a letter is a number. the letter comes back only when someone draws it again.

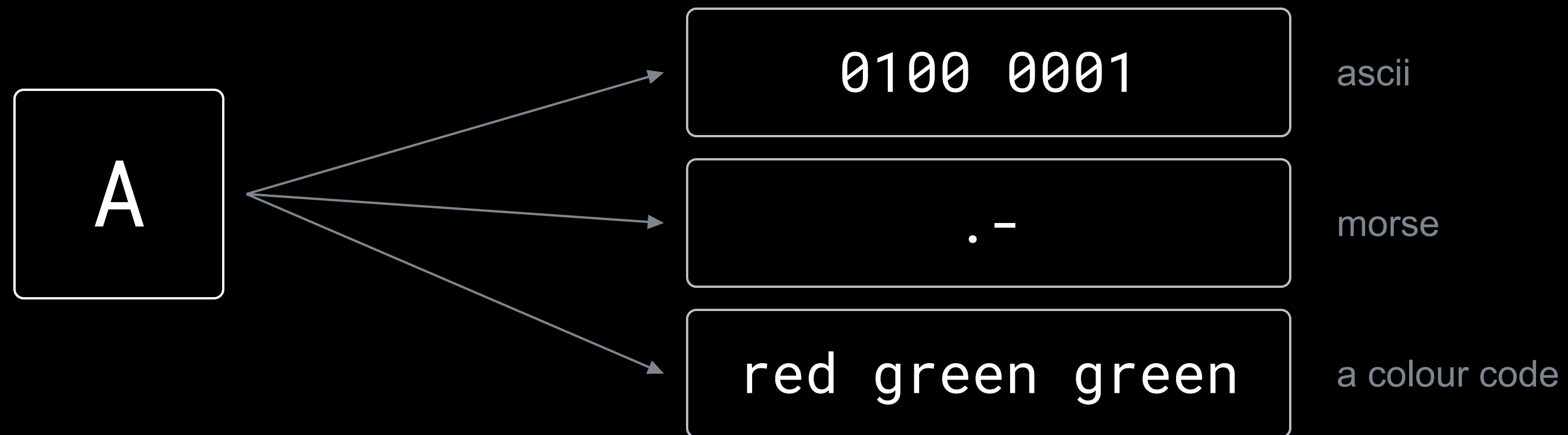
what a code system is

character		number	bits
A	→	65	0100 0001
B	→	66	0100 0010
C	→	67	0100 0011
a	→	97	0110 0001

a code is not encryption, not compression, and not source code

a letter is a number. the letter comes back only when someone draws it again.

the same letter, three code systems



agreed, not given by nature. that is why there are several.

when the table is too small

does ascii have it?

A	z	7	?	ä	€	π
✓	✓	✓	✓	✗	✗	✗

utf-8 then writes those numbers down: common ones in one byte, rare ones in up to four

one table was too small. so a bigger one was agreed.

when the table is too small

does ascii have it?

A	z	7	?	ä	€	π
✓	✓	✓	✓	✗	✗	✗

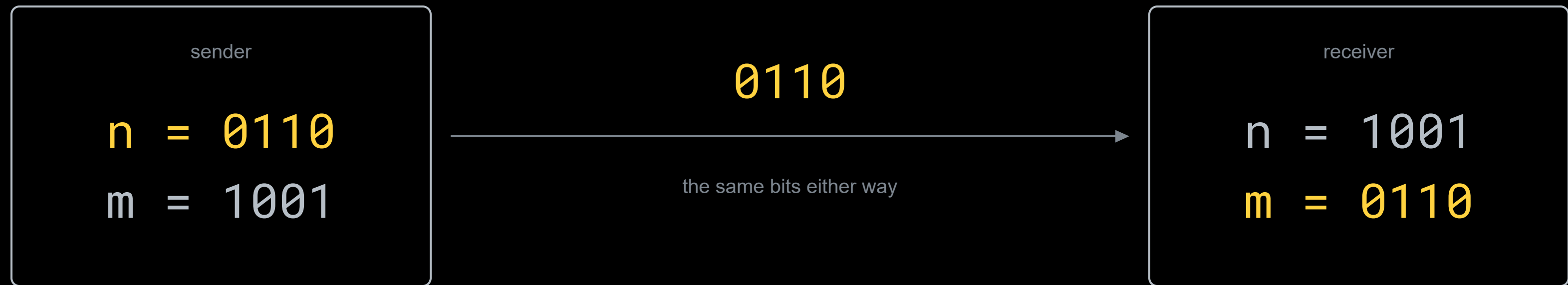
unicode: a number for every one of them

65 122 55 63 228 8364 960

utf-8 then writes those numbers down: common ones in one byte, rare ones in up to four

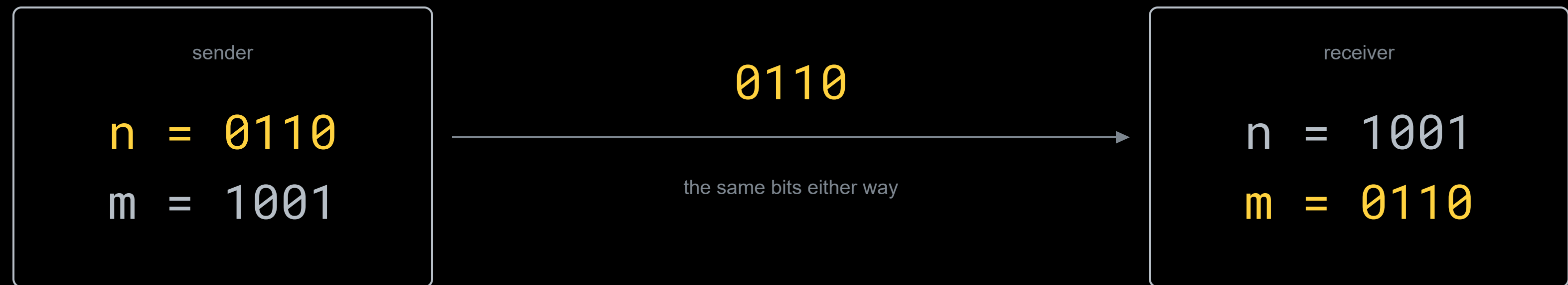
one table was too small. so a bigger one was agreed.

two tables, one message



no error message. just clean nonsense.

two tables, one message

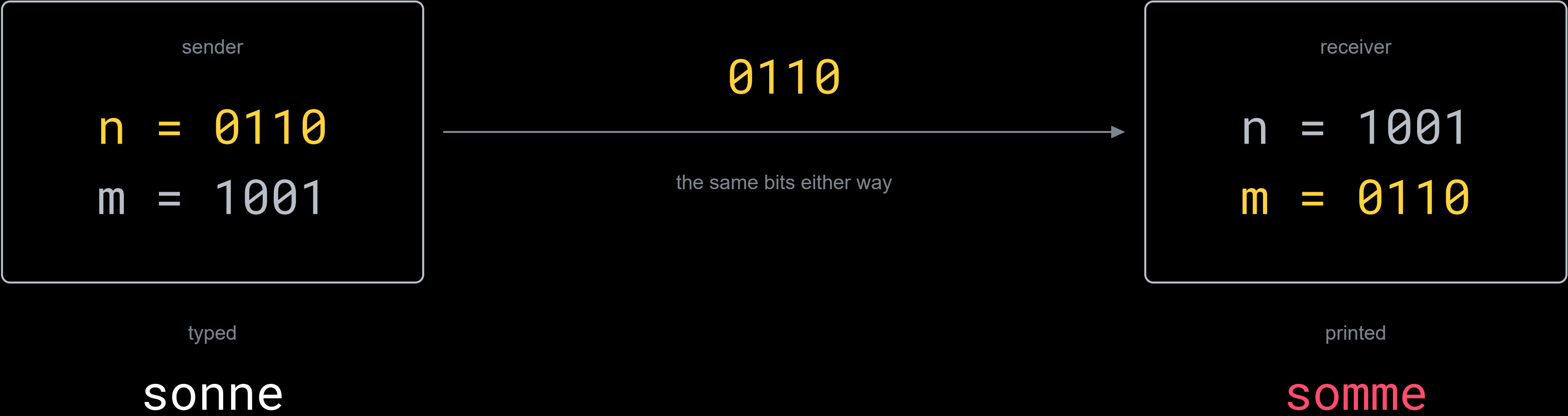


typed

sonne

no error message. just clean nonsense.

two tables, one message



no error message. just clean nonsense.

part 2

build your own codebook

how many positions per letter?

your alphabet



26 letters to encode

1 position

4 codes



not enough for 26

2 positions

$4 \times 4 = 16$

3 positions

$4 \times 4 \times 4 = 64$

26 letters, 3 colour positions, 38 codes to spare.

how many positions per letter?



26 letters to encode

1 position

4 codes



not enough for 26

2 positions

$4 \times 4 = 16$



not enough for 26

3 positions

$4 \times 4 \times 4 = 64$

26 letters, 3 colour positions, 38 codes to spare.

how many positions per letter?



26 letters to encode

1 position

4 codes



not enough for 26

2 positions

4 x 4 = 16



not enough for 26

3 positions

4 x 4 x 4 = 64



enough for 26

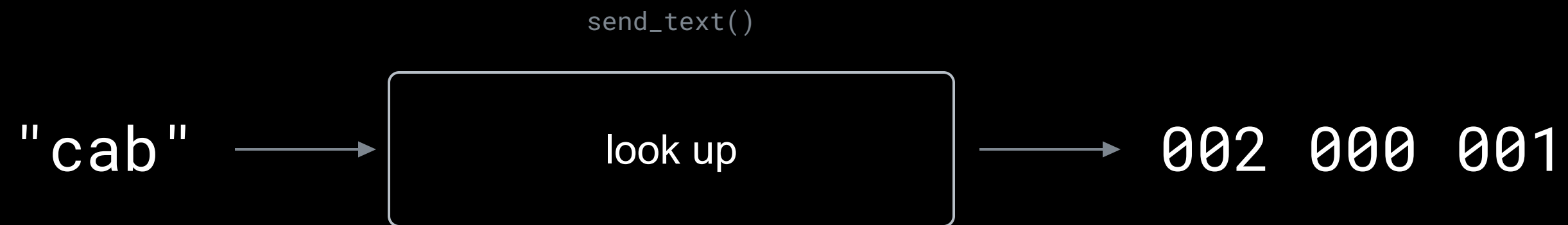
26 letters, 3 colour positions, 38 codes to spare.

the codebook

a 000	b 001	c 002	d 003	e 010	f 011	g 012	h 013
i 020	j 021	k 022	l 023	m 030	n 031	o 032	p 033
q 100	r 101	s 102	t 103	u 110	v 111	w 112	x 113
y 120	z 121	⌋ 122	. 123	0 130	1 131	2 132	3 133
4 200	5 201	6 202	7 203	8 210	9 211	 212	 213
 220	 221	 222	 223	 230	 231	 232	 233
 300	 301	 302	 303	 310	 311	 312	 313
 320	 321	 322	 323	 330	 331	 332	 333

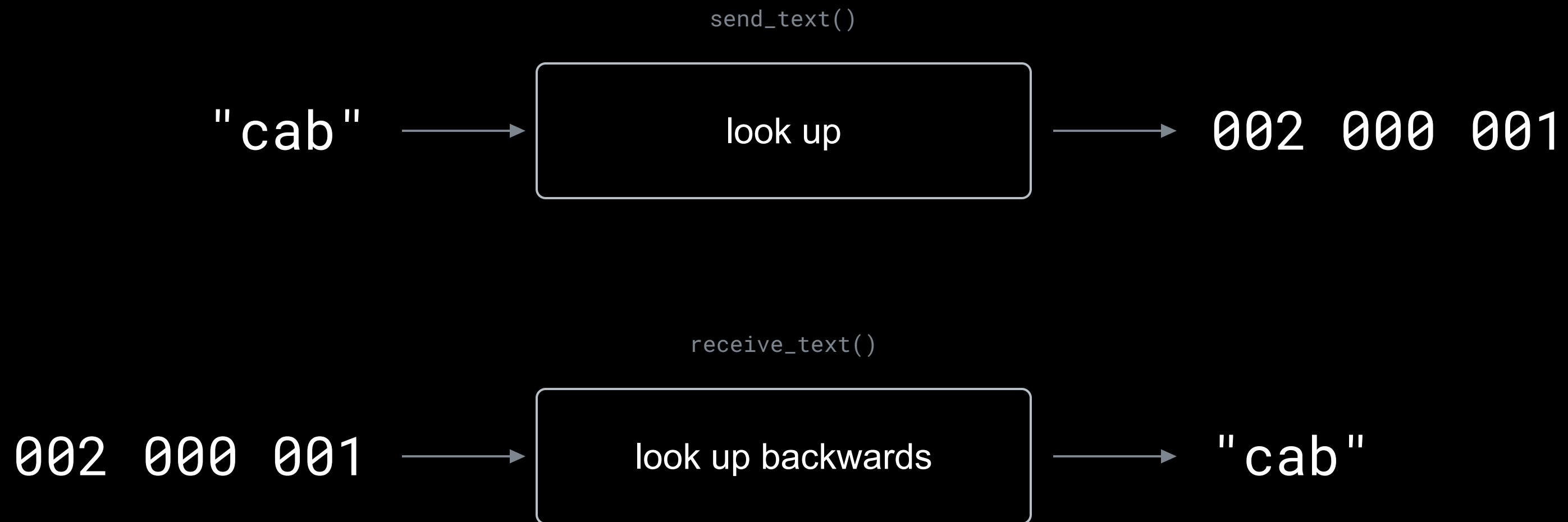
the spare codes are stock, not waste.

the same table, both directions



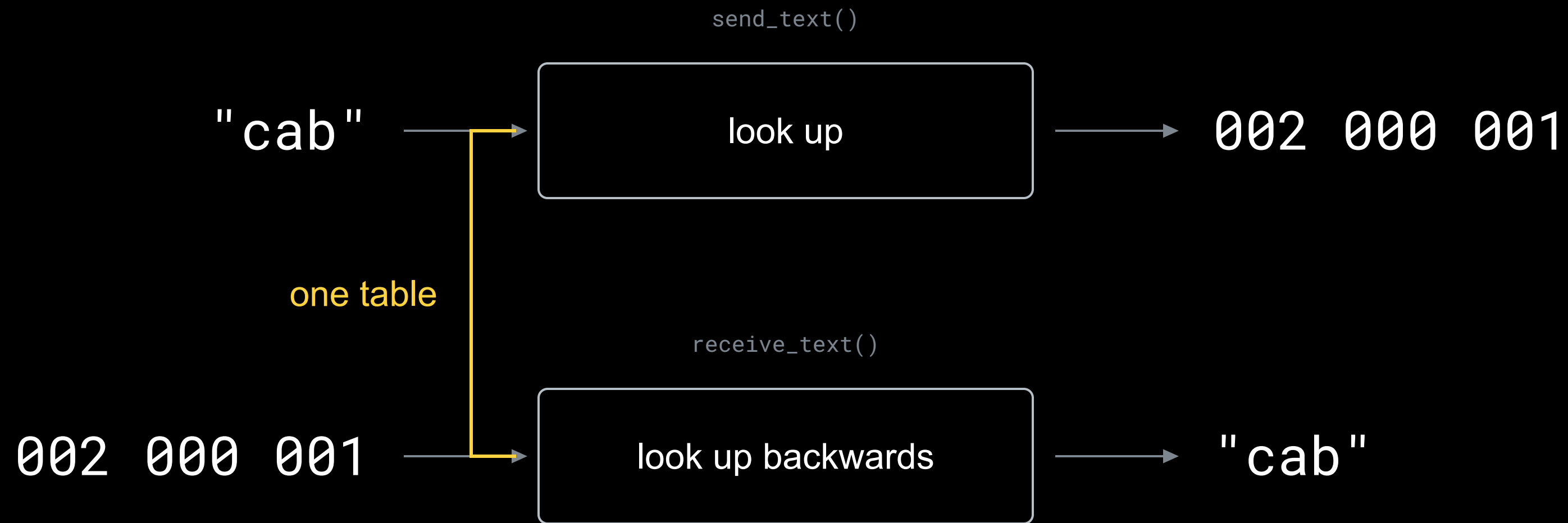
decoding is the same table, read the other way.

the same table, both directions



decoding is the same table, read the other way.

the same table, both directions



decoding is the same table, read the other way.

what is not in the table

send_text("guten morgen")

your codebook has a to z and nothing else

g	u	t	e	n	space	m	o	r	g	e	n
✓	✓	✓	✓	✓	?	✓	✓	✓	✓	✓	✓

three ways out, and you have to pick one

add it
to the table

drop it
before sending

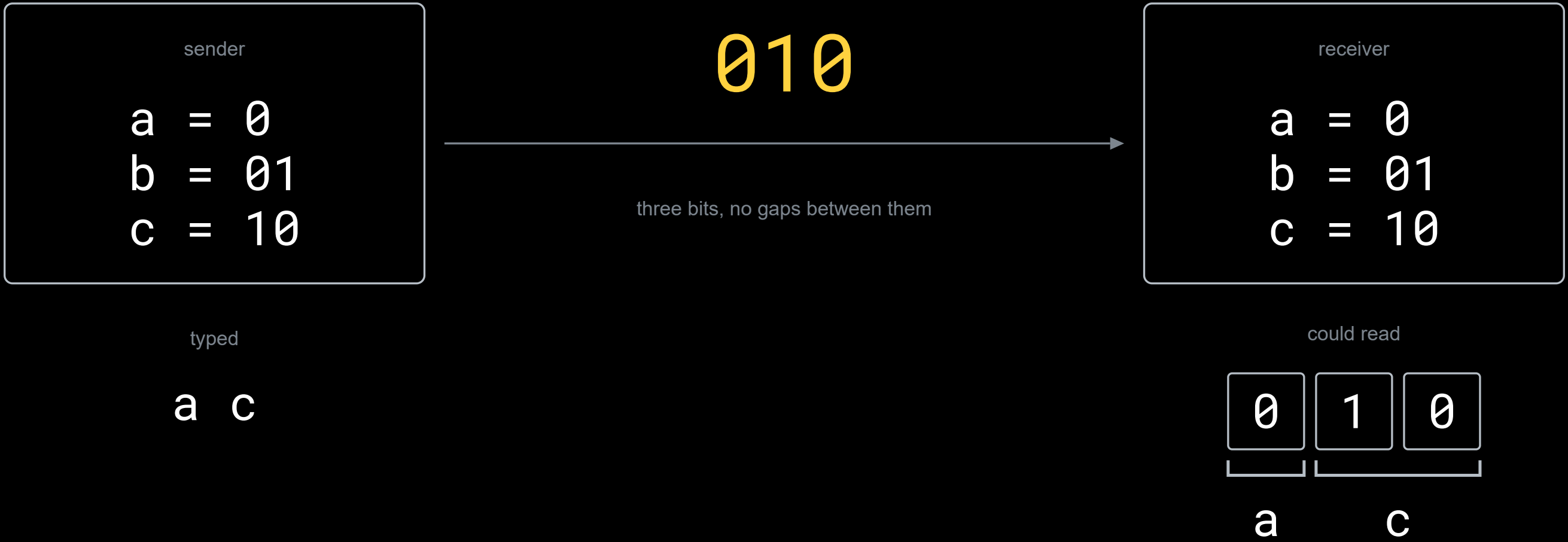
stop
with a message

something happens either way. decide which.

part 3

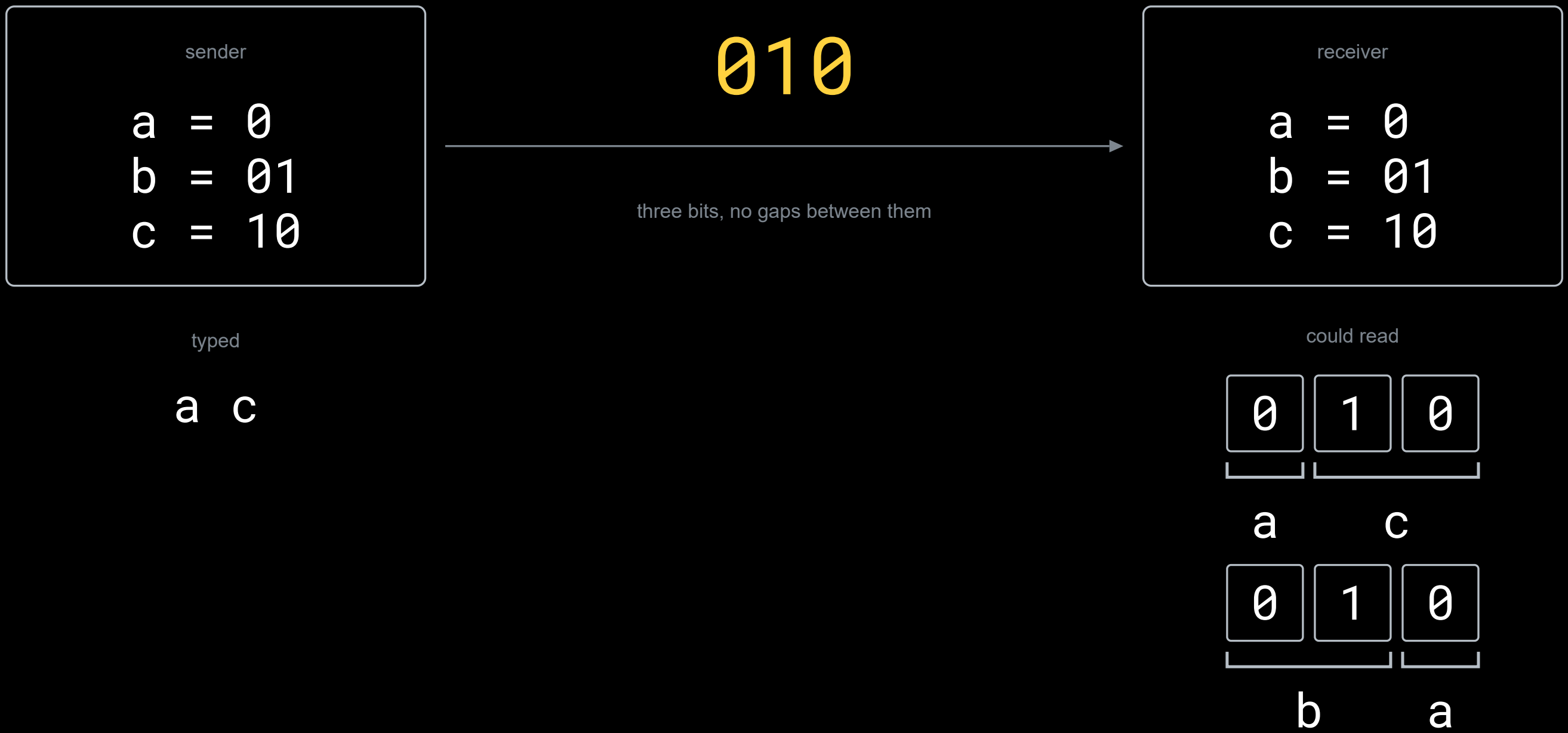
where does a character end?

a code that does not work



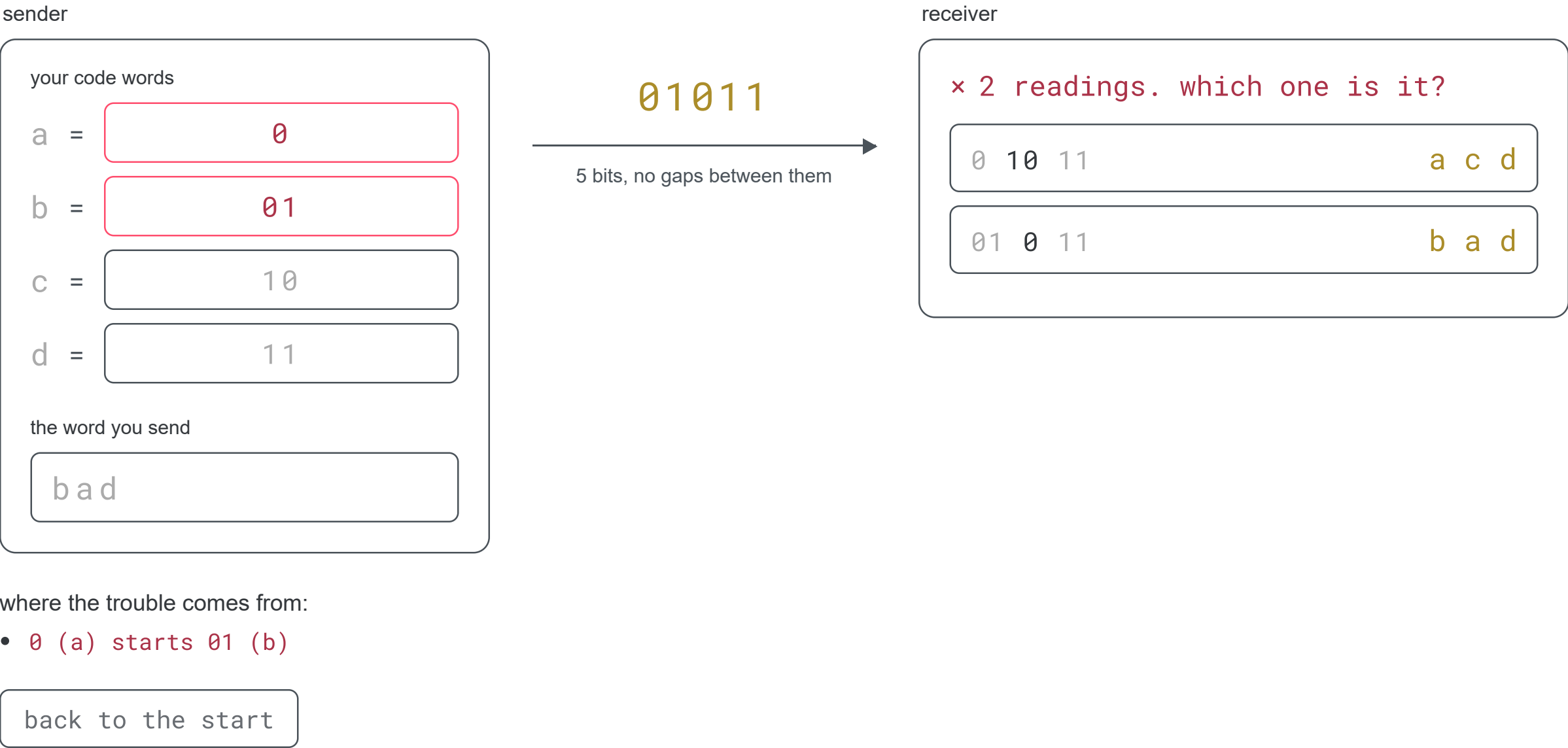
same bits, two readings. the receiver cannot see the boundary.

a code that does not work



same bits, two readings. the receiver cannot see the boundary.

try it: the prefix trap



three ways out

fixed length

a = 00

b = 01

c = 10



count instead of guessing

a separator

a = .-

b = -...

c = -.-.



morse pauses between letters

no code word
starts another

a = 0

b = 10

c = 110



read until it matches

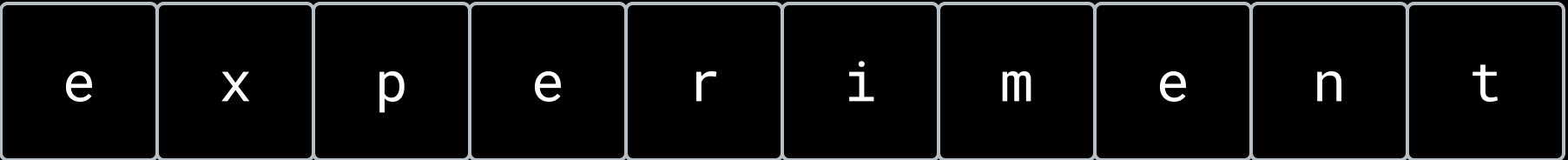
three answers to one question. all of them work.

what fixed length costs

the same word in two codes of your own

fixed

three positions for every letter

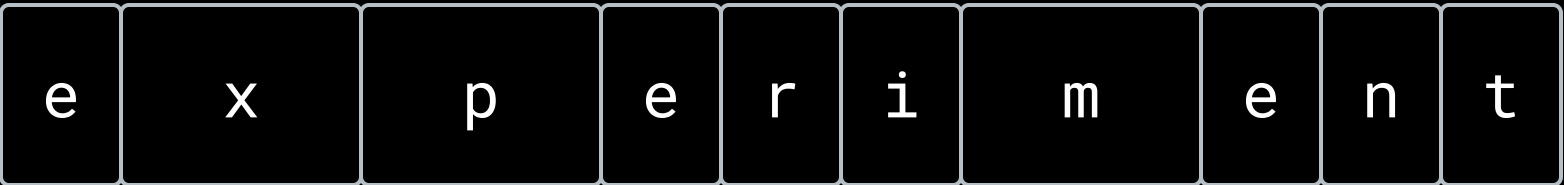


30 positions

3.0 per letter

variable

two for the common ones, four for the rest



26 positions

2.6 per letter

the ten most common letters in german and english: e n i s r a t d h u

shorter on average, not in every case.

part 4

the same bits, other meanings

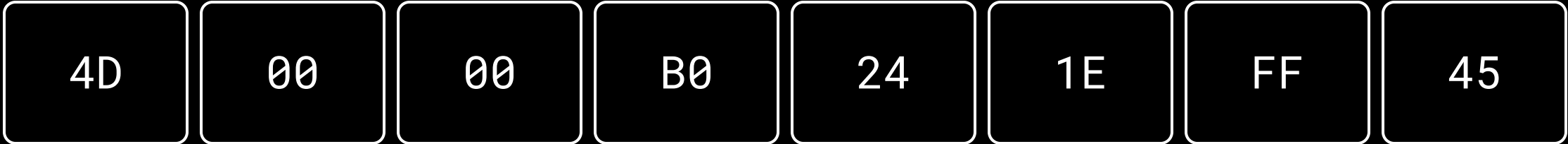
one byte sequence, three readings



the photo

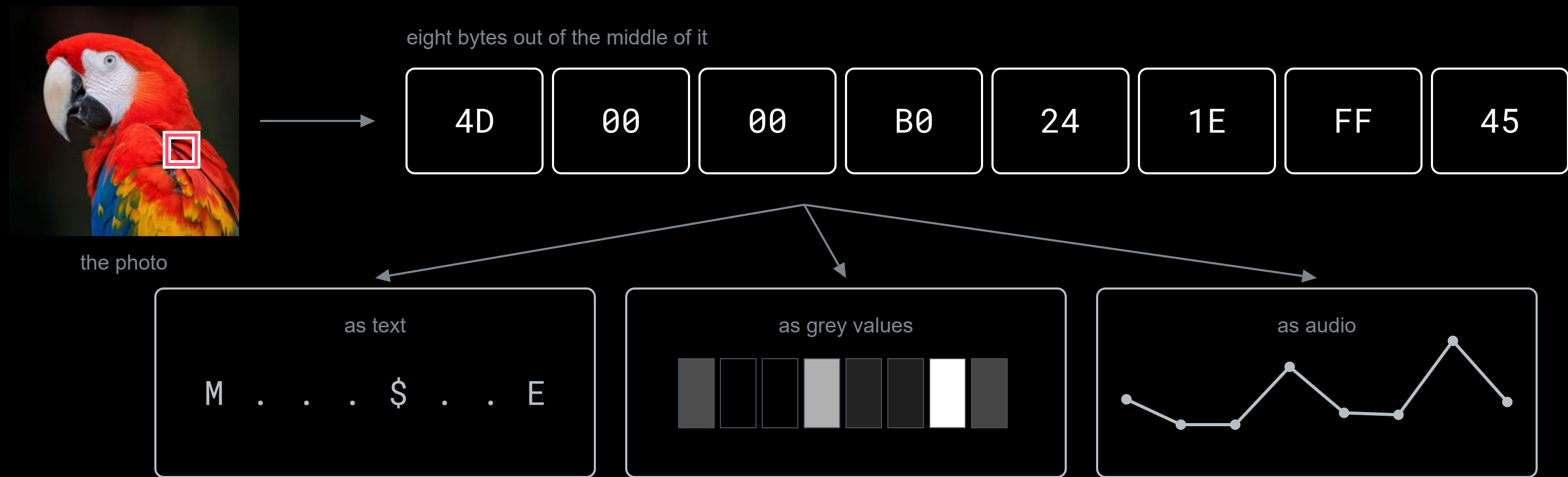


eight bytes out of the middle of it



the bytes do not say which reading is right.

one byte sequence, three readings



the bytes do not say which reading is right.

one pixel, three numbers



one pixel of it



red		236
green		171
blue		3

the same colour, written as hex: two places per byte

#ECAB03

the same three numbers your sender writes: `led.setColor(236, 171, 3)`

try it: pixel painter

	binary	hex
☐	0000	00
☐	0110	66
☐	1111	FF
☐	1111	FF
☐	0111	7E
☐	0011	3C
☐	0001	18
☐	0000	00

64 pixels = 64 bits = 8 bytes · 32 pixels on

start from: ☐ ☐ ☐ ☐ ☐

break a byte

flip one bit

repair

invert

clear

who says which reading?

the file extension

.txt .png .wav

a header in the file

the first bytes say it

a type field in your frame

you decide what it says

challenge 3

the type is another agreement.

part 5

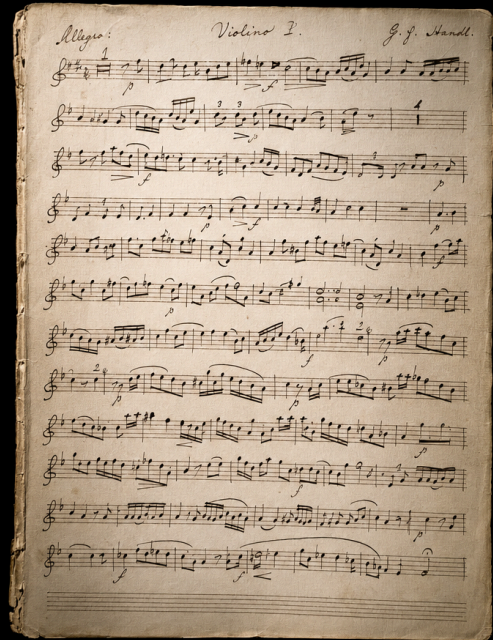
codes without computers

four agreements, no computer



barcode

bar widths → digits



sheet music

position → pitch and length



braille

six dots → a character



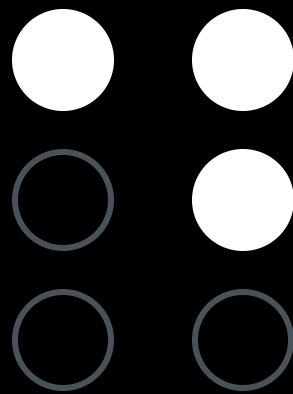
dna

three bases → an amino acid

none of them needs a computer. all of them need an agreement.

the same calculation, three times

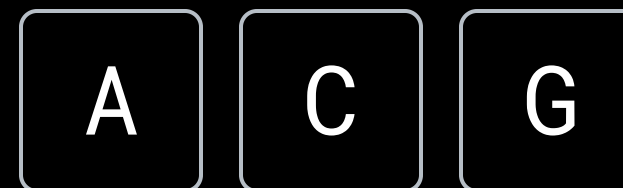
braille



2 states, 6 positions

$$2^6 = 64$$

dna



4 bases, 3 positions

$$4^3 = 64$$

your alphabet



4 colours, 3 positions

$$4^3 = 64$$

braille, dna and your alphabet all landed on 64.

write it down.
twice.

